PROJECT & OPERATIONAL KNOWLEDGE

Project Technical Specification: Enterprise Knowledge Assistant

System Architecture, Semantic Chunking, Hybrid Vector Store, and Grounded RAG Pipeline

System: Python 3.11 / Django / pgvector **Version:** 2.4 **Owner:** AI Architecture Core **Status:** Active Production

1. System Overview and Business Context

The **Enterprise Knowledge Assistant (EKA)** is Apex Global Technologies' internal conversational intelligence platform. EKA ingests, indexes, and queries our vast institutional corpus—comprising HR policies, IT security standards, engineering documentation, operational SOPs, and system postmortems—enabling employees to obtain authoritative, grounded, and verified answers with precise source citations.

2. End-to-End Retrieval-Augmented Generation (RAG) Architecture

Core Query Solution: Technologies & RAG Pipeline Mechanics

Which technologies does EKA use, and how does its RAG pipeline work?

EKA is constructed on a high-performance stack: **Python 3.11, Django 4.2 REST Framework, Celery + Redis, PostgreSQL 15 with pgvector, and Anthropic Claude 3.5 Sonnet / OpenAI text-embedding-3-large**. Its operational RAG pipeline executes in six discrete phases:

1. **Document Ingestion & Parsing:** Files (PDF, Markdown) uploaded via API are parsed using PyPDFium2 and Markdown-it. Tables and structural headings are normalized into Markdown syntax.
2. **Semantic Chunking:** Text is partitioned into **512-token chunks with 50-token sliding overlap**, preserving document metadata (title, section headers, doc_id, category).
3. **Vector Embedding:** Each chunk is converted into a **3072-dimensional vector** using OpenAI's `text-embedding-3-large` model.
4. **Hybrid Search (Dense + Sparse):** Queries trigger parallel retrieval across PostgreSQL 15:
   - **Dense Vector Search:** Cosine similarity search using `pgvector` with HNSW indexing.
   - **Sparse Lexical Search:** Keyword search using PostgreSQL full-text search (tsvector with BM25 ranking).
   - **Reciprocal Rank Fusion (RRF):** Merges top-50 vector and top-50 keyword matches into a unified candidate pool.
5. **Cross-Encoder Re-Ranking:** The candidate pool is evaluated by `BAAI/bge-reranker-large`, returning the **top-5 highest-scoring relevant chunks**.
6. **Grounded Generation with Strict Citations:** An LLM prompt injects the top-5 chunks into Claude 3.5 Sonnet (temperature 0.1). The model must cite exact sources (`[Document_Name, Section, Page]`) for every claim. If ungrounded, it states that information is not available.

3. Technology Stack Breakdown

Component Layer	Selected Technologies	Architecture Rationale
Core Application	Python 3.11, Django 4.2 LTS, DRF	Enterprise reliability, built-in ORM security, robust REST tooling.
Async Task Queue	Celery 5.3, Redis 7 (ElastiCache)	Decoupled document chunking and vector embedding generation.

Component Layer	Selected Technologies	Architecture Rationale
Vector & Data Store	PostgreSQL 15 + pgvector extension	Unified relational metadata and high-dimensional vector search.
Reranking Model	BAAI/bge-reranker-large (Triton)	High semantic precision; reduces hallucination by 94%.
Frontend Interface	Next.js 14, Tailwind CSS, TypeScript	Real-time streaming responses, citation popovers, feedback UI.

4. Proposed Future Architectural Enhancements

- **GraphRAG Integration:** Extracting entity-relationship knowledge graphs using Neo4j to resolve complex multi-hop organizational queries.
- **Multimodal Ingestion:** Ingestion of architectural diagrams and flowchart images via vision-language models.